

Data Science Cheat Sheet

The 20% of functions that cover 80% of real-world usage

1. NumPy

Import: import numpy as np — arrays are faster than lists; vectorize everything.

1.1 Array Creation

```
np.array([1, 2, 3])
np.zeros((3, 4)) # 3x4 zeros
np.ones((2, 3)) # 2x3 ones
np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5) # 5 evenly spaced values
np.random.randn(3, 3) # 3x3 normal distribution
```

1.2 Shape & Dtype Inspection

```
a.shape # (rows, cols)
a.ndim # number of dimensions
a.dtype # int64, float32, ...
a.astype(float) # cast to float
a.reshape(6, 1) # new shape (same data)
a.flatten() # always returns a copy
```

1.3 Indexing & Slicing

```
a[0] # first row
a[1, 2] # row 1, col 2
a[0:3, 1:4] # rows 0-2, cols 1-3
a[:, 0] # all rows, col 0
a[-1] # last row
# Boolean & fancy indexing
a[a > 0] # all positive values
a[(a > 0) & (a < 5)]
a[[0, 2, 4]] # rows 0, 2, 4
np.where(a > 0, a, 0) # replace negatives with 0
```

1.4 Math & Statistics

```
a.sum() # total sum
a.sum(axis=0) # column sums
a.mean() a.std() a.var()
a.min() a.max()
a.argmin() # index of minimum
a.cumsum() # cumulative sum
# Linear algebra
np.dot(A, B) # matrix multiply
A @ B # same, Python 3.5+
```

```
np.linalg.inv(A)
np.linalg.eig(A) # eigenvalues / vectors
np.linalg.norm(v) # L2 norm
```

2. Pandas

Import: import pandas as pd — DataFrames are the workbench; think in columns, not loops.

2.1 Reading Data

```
pd.read_csv('file.csv')
pd.read_csv('f.csv', index_col=0, parse_dates=['date'])
pd.read_excel('file.xlsx', sheet_name='Sheet1')
pd.read_json('file.json')
pd.read_sql(query, connection)
```

2.2 Quick Inspection

```
df.head(5) df.tail(3)
df.info() # dtypes + null counts
df.describe() # numeric statistics
df.value_counts()
df.shape df.columns df.dtypes
```

2.3 Selection & Filtering

```
df.loc[0:5, ['a', 'b']] # label-based
df.iloc[0:5, 0:2] # position-based
df.loc[df['col'] > 0] # filter rows
df['col'].isin(['a', 'b'])
```

2.4 Missing Values

```
df.isnull().sum() # count NaN per column
df.dropna(subset=['col'])
df.fillna(0) # fill with 0
df['col'].fillna(df['col'].median())
```

2.5 GroupBy

```
df.groupby('cat')['val'].mean()
df.groupby('cat').agg(
    n=('id', 'count'),
    avg=('val', 'mean')
)
df.pivot_table(values='v', index='r', columns='c', aggfunc='mean')
```

2.6 Merge & Join

```
pd.merge(df1, df2, on='id', how='left')
pd.concat([df1, df2], ignore_index=True)
# how options: 'inner', 'left', 'right', 'outer'
```

2.7 Apply & Map

```
df['col'].apply(lambda x: x * 2)
```

```
df.apply(np.mean, axis=0)
df['cat'].map({'a': 1, 'b': 2})
pd.get_dummies(df['col'])
```

3. Matplotlib

Import: import matplotlib.pyplot as plt — always end with plt.tight_layout() and plt.show().

3.1 Core Template (use every time)

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y, color='steelblue', label='line')
ax.set_title('Title')
ax.set_xlabel('X') ax.set_ylabel('Y')
ax.legend()
plt.tight_layout()
plt.show() # or: plt.savefig('fig.png', dpi=150)
```

3.2 Plot Types

```
ax.plot(x, y) # line chart
ax.scatter(x, y, c=z) # scatter with color
ax.bar(cats, vals) # vertical bar chart
ax.barh(cats, vals) # horizontal bar chart
ax.hist(data, bins=30) # histogram
```

3.3 Multiple Subplots

```
fig, axes = plt.subplots(2, 3, figsize=(12, 6))
# Access via: axes[0, 0], axes[0, 1], ...
fig.suptitle('Dashboard', y=1.02)
plt.subplots_adjust(hspace=0.4)
```

3.4 Styling

```
ax.set_xlim(0, 100)
ax.set_xticks([0, 25, 50, 75, 100])
ax.grid(True, linestyle='--', alpha=0.5)
ax.axhline(y=0, color='red')
ax.annotate('Peak', xy=(x, y), xytext=(x+1, y+2),
            arrowprops=dict(arrowstyle='->'))
plt.style.use('seaborn-v0_8-whitegrid')
plt.style.use('ggplot')
ax.scatter(x, y, c=z, cmap='viridis')
```

4. Seaborn

Import: import seaborn as sns — built on matplotlib; pass a DataFrame + column names. Set theme once:
`sns.set_theme(style='whitegrid')`

4.1 Distribution Plots

```
sns.histplot(df, x='age', hue='gender', kde=True)
```

```
sns.kdeplot(df, x='score', fill=True)
sns.boxplot(df, x='cat', y='val')
sns.violinplot(df, x='cat', y='val', inner='box')
```

4.2 Relationship Plots

```
sns.scatterplot(df, x='x', y='y', hue='cat', size='val')
sns.lineplot(df, x='date', y='val', hue='group')
sns.regplot(df, x='x', y='y') # scatter + regression line
```

4.3 Heatmap (Correlation Matrix)

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr,
            annot=True, fmt='.2f',
            cmap='coolwarm', vmin=-1, vmax=1,
            square=True, linewidths=0.5)
```

4.4 Categorical Plots

```
sns.barplot(df, x='cat', y='val', errorbar='sd')
sns.countplot(df, x='cat', hue='group')
sns.stripplot(df, x='cat', y='val', jitter=True)
```

4.5 FacetGrid & Pairplot

```
g = sns.FacetGrid(df, col='group', hue='cat', height=4)
g.map_dataframe(sns.scatterplot, x='x', y='y')
g.add_legend()
sns.pairplot(df, hue='target', diag_kind='kde')
```

4.6 hue / size / style (free 3rd dimension)

```
sns.scatterplot(df, x='income', y='spend',
                hue='region', # color encodes category
                size='age', # marker size encodes numeric
                style='gender', # marker style encodes another category
                palette='tab10')
# Save after any seaborn call:
plt.savefig('plot.png', dpi=150, bbox_inches='tight')
```